T04

Permissions : Glob Patterns & Whitelist

Technique Intermediate

Contrôler précisément les outils accessibles avec des patterns glob

Deux formats de patterns

Format simple : correspond au nom d'outil ou à une commande Bash.

```
"permissions": {
  "allow": ["Bash(git *)", "Bash(pnpm *)", "Read", "Edit"],
  "deny":  ["Bash(rm -rf *)", "Bash(sudo *)", "Bash(npm publish)"]
}
```

Patterns de chemins : Read, Edit et Write acceptent directement le pattern du chemin. Ne pas ajouter `file_path:` ; ce qualificatif est invalide pour ces outils.

```
"permissions": {
  "deny": [
    "Read(*.env*)",
    "Edit(*.pem)",
    "Write(*.key)",
    "Read(*credentials*)"
  ]
}
```

Syntaxe glob

Pattern	Ce qui correspond
`Bash(git *)`	Toute commande git
`Bash(npm test*)`	`npm test` , `npm test:watch`
`Read(*.env*)`	`.env` , `.env.local` , `.env.prod`
`Edit(./src/**)`	Tous les fichiers sous `src/`
`Write(*.key)`	Tout fichier `.key`

Niveaux progressifs

```
// Niveau 1 - Préautoriser les outils de lecture
{ "permissions": { "allow": ["Read", "Grep", "Glob"] } }

// Niveau 2 - Git + gestionnaire de paquets
{ "permissions": { "allow": ["Read", "Grep", "Glob",
   "Bash(git *)", "Bash(pnpm *)"] } }

// Niveau 3 - Développement complet
{ "permissions": { "allow": ["Read", "Grep", "Glob",
   "Edit", "Write", "Bash(git *)", "Bash(pnpm *)"] } }
```

Comportements des trois catégories

Catégorie	Comportement
`allow`	Auto-approuvé, sans demande
`deny`	Bloqué complètement
`ask`	Demande de confirmation
(défaut)	Mode de permission par défaut

Limitation connue

Une règle de chemin restreint les accès correspondants via les outils ; elle ne retire pas un secret déjà collé dans un prompt ou renvoyé par un autre outil. Garder les données sensibles hors de la portée d'accès du processus et examiner ensemble les contrôles du shell, de MCP et du sandbox.

Où définir les permissions

```
~/.claude/settings.json       (global, tous les projets)
.claude/settings.json         (projet, partagé avec l'équipe)
.claude/settings.local.json   (machine locale, gitignore)
```

Évaluation des règles : les règles deny de tout scope priment sur ask et allow. Un allow local ne peut pas contourner un deny géré ou utilisateur. Les paramètres locaux servent aux préférences propres au projet dans ces limites.